Lab 4: Networking in Cloud

A **load balancer** is a critical component in modern networking and cloud infrastructure that helps distribute incoming network traffic across multiple servers or resources. Its primary goal is to ensure that no single server or resource is overwhelmed with too much traffic, thus improving the **availability, reliability, and scalability** of applications or services. Load balancers play an essential role in networking by optimizing traffic flow, ensuring fault tolerance, and improving overall application performance.

Load balancers operate at the **network layer** of the OSI model (Layer 4) or the **application layer** (Layer 7), depending on the type of load balancer. Their job is to distribute incoming client requests to multiple backend servers (also known as "pool" or "targets"), ensuring that no single server becomes a bottleneck.

- **Layer 4 Load Balancer (Network Layer)**: Routes traffic based on **IP address** and **port number**.
- **Layer 7 Load Balancer (Application Layer)**: Routes traffic based on more advanced criteria, such as **HTTP headers**, **cookies**, **URL paths**, etc.

Objective:

- Implement a Load Balancer to distribute traffic across 2 EC2 instances. Test by accessing the Load Balancer's URL and verifying that the traffic is routed correctly to the instances.

Submission:

- Screenshots important steps
- In your lab setup, what would happen if one of the backend servers in your load balancer pool becomes unavailable (e.g., crashes)?
- Why is load balancing important for highly available web applications?

Bonus Exercises

Auto-scaling group setup

- Terminate all your existing EC2 instances
- Create a launch template with your working EC2 configuration and user data

- Create an auto-scaling group with the above launch template. Set minimum 2, maximum 4, desired 2. Use the existing target group from the previous session or create a new one.
- Add health check path: /index.html in target group
    - Using /index.html specifically ensures:
        - You're checking a real file
        - You avoid redirects or default document issues (some servers handle / differently)
        - It's a more **predictable and controlled** health endpoint
        - You could also use a **custom health check endpoint**, like /health, if you're running an application that exposes one.

Monitor with CloudWatch

- Create alarms for:
    - **High request count** – Metric: CPU Utilization; Condition: Greater than 70% for 2 consecutive periods of 1 minute
    - **Instances in Unhealthy state**
- Test CPU Utilization Alarm (on EC2 or ASG)
    - Select EC2 instance or Auto Scaling Group
    - Choose Metric: CPU Utilization
    - Set condition: Greater than 70% for 2 consecutive periods of 1 minute
    - SSH into the EC2 instance and run:
      sudo yum install -y stress
      stress --cpu 2 --timeout 120 # stresses 2 cores for 2 minutes
    - Wait 2–5 minutes and monitor the alarm status in CloudWatch.

Sticky session

- Enable stickiness in your target group
- Set it to 1 minute for testing
- Refresh ALB url multiple times — you'll be consistently served from the same instance due to sticky sessions.
- Observe behavior of ALB after 1 minute

Submission

- Screenshots important steps of all exercises
- What is the purpose of setting up ASG?
- Why should you set up clover alarm?
- Why and when to use sticky session?

https://www.youtube.com/watch?v=KNr3Kq7cah8